



Instituto Tecnológico Nacional de México

Instituto Tecnológico de Saltillo



Cómputo en la Nube

Miguel Salazar del Bosque

2.3. Explorando Github

Roberto Aram López Rodríguez

No. Control: 22050788

Mariana Elizabet Mota Piña

No.Control:22050725

25 -Febrero- 2026

Introducción

El uso de plataformas de control de versiones como GitHub se ha vuelto fundamental en el desarrollo de software moderno. Estas herramientas no solo permiten almacenar código, sino también gestionar cambios, colaborar en equipo y mantener un historial organizado del proyecto. En el ámbito académico, explorar GitHub permite comprender cómo trabajan los equipos de desarrollo en entornos reales, aplicando prácticas de control de versiones, documentación y trabajo colaborativo.

Durante esta actividad se realizó una exploración práctica de la plataforma GitHub, creando repositorios, subiendo archivos, analizando sus herramientas de colaboración y revisando funciones como pull requests, issues y GitHub Actions. El objetivo fue relacionar el uso de la plataforma con situaciones reales de trabajo en equipo dentro del desarrollo de software y la computación en la nube.

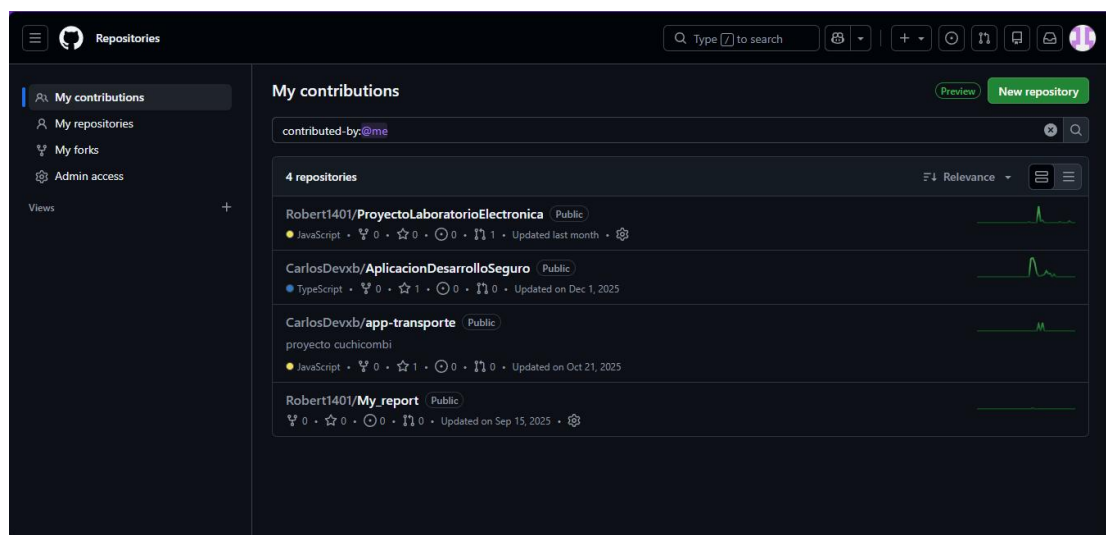
1. Diferencias entre repositorio público y privado

Durante la exploración de GitHub identifiqué que la principal diferencia entre un repositorio público y uno privado radica en el nivel de visibilidad y control del acceso.

Un repositorio público permite que cualquier persona pueda visualizar el código, descargarlo y, si el proyecto lo permite, proponer mejoras. Este tipo de repositorios es ideal cuando se desea compartir conocimiento, construir un portafolio profesional o colaborar en proyectos de código abierto.

En cambio, un repositorio privado restringe el acceso únicamente a los colaboradores autorizados. Esto resulta conveniente cuando el proyecto contiene información sensible, está en etapa de desarrollo temprano o forma parte de un entorno empresarial donde la confidencialidad es prioritaria.

Dentro de un entorno educativo, resulta conveniente trabajar inicialmente en un repositorio privado para evitar modificaciones externas o plagio, y posteriormente publicarlo como evidencia del trabajo realizado y del dominio de herramientas de control de versiones.



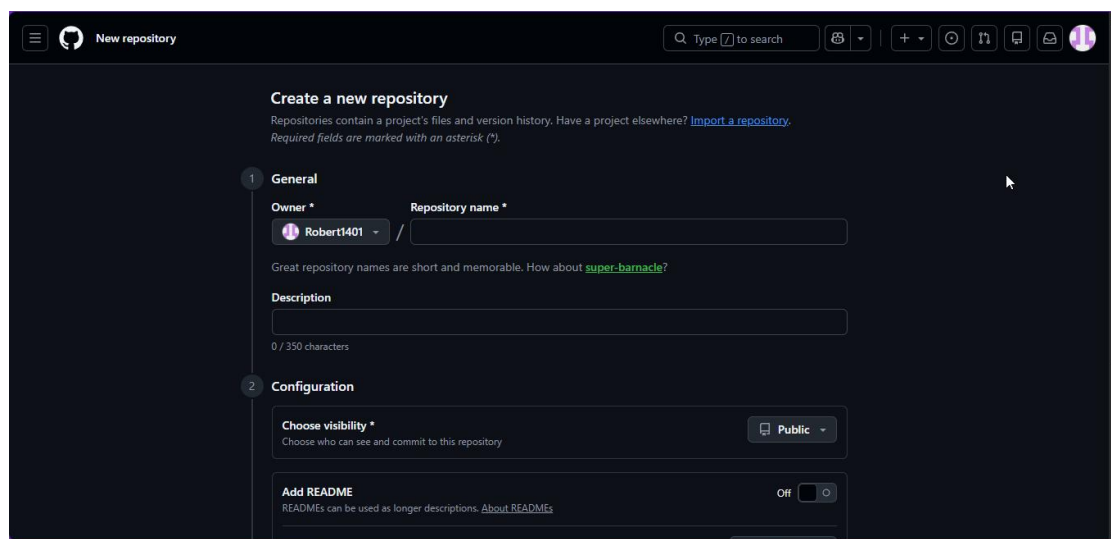
2. Proceso para crear mi primer repositorio y subir un archivo

Para crear mi primer repositorio realicé el siguiente procedimiento:

Primero inicié sesión en GitHub y seleccioné la opción de crear un nuevo repositorio. Asigné un nombre representativo relacionado con la actividad, agregué una breve descripción y seleccioné la visibilidad deseada. Activé la creación automática del archivo README para documentar el propósito inicial del proyecto.

Posteriormente, para subir un archivo desde mi equipo local, utilicé la opción de carga directa en la interfaz web. Seleccioné el archivo, redacté un mensaje descriptivo del cambio realizado y confirmé la acción mediante un commit.

Este proceso me permitió comprender que GitHub no solo almacena archivos, sino que registra versiones estructuradas del desarrollo.

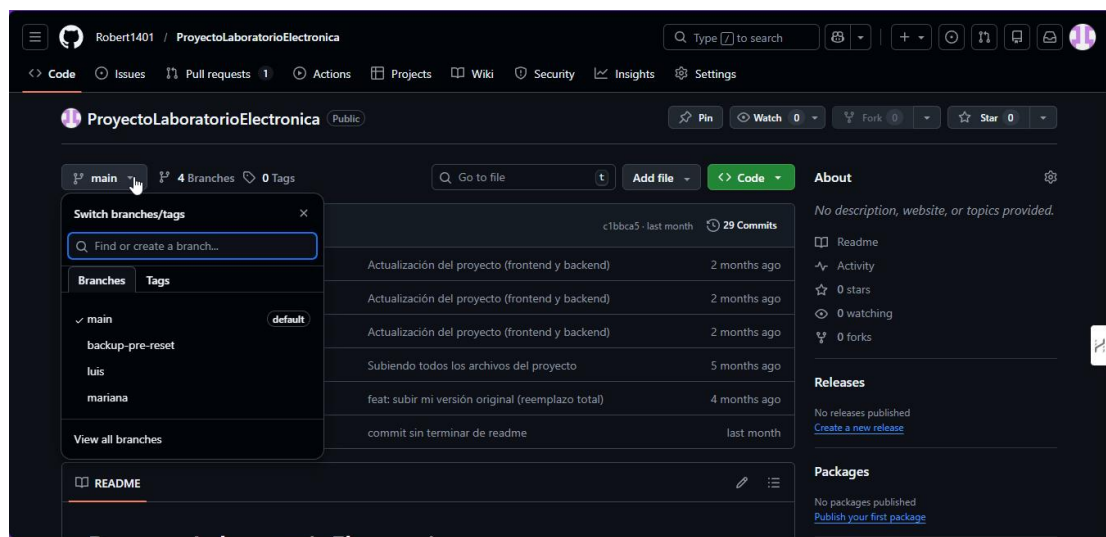


3. ¿Qué son los commits y por qué son importantes los mensajes claros?

Un commit representa una unidad de cambio dentro del historial del proyecto. Cada commit actúa como un punto de control que guarda el estado exacto del código en un momento determinado.

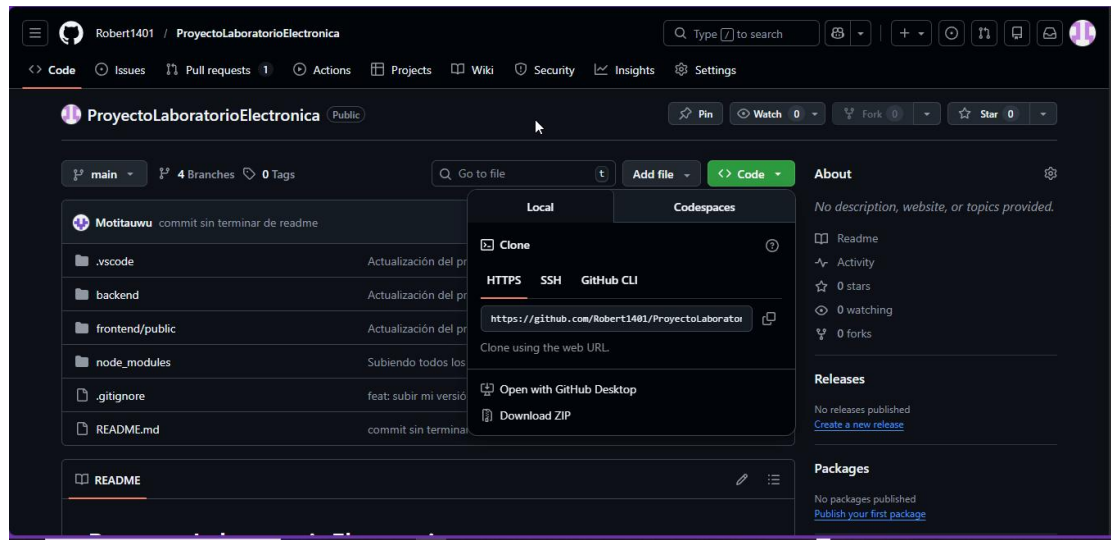
Comprendí que la redacción de mensajes claros es fundamental porque facilita el rastreo de modificaciones. En proyectos colaborativos, un mensaje ambiguo puede generar confusión, mientras que uno preciso permite identificar rápidamente qué funcionalidad se agregó, corrigió o mejoró.

Dentro de un ambiente laboral, esto mejora la capacidad de dar mantenimiento al sistema y permite llevar un control claro de las modificaciones realizadas a lo largo del desarrollo.



4. Proceso de clonar un repositorio y ventajas

Para clonar un repositorio copié la URL proporcionada por GitHub y utilicé la terminal para ejecutar el comando correspondiente. Esto generó una copia local completa del proyecto en mi equipo.



La principal ventaja de este procedimiento es que permite trabajar de manera local sin depender constantemente de conexión a internet. Además, facilita la experimentación, pruebas y control de versiones antes de sincronizar los cambios con el repositorio remoto.

Desde el punto de vista del trabajo colaborativo, clonar garantiza que todos los integrantes partan de una base común y actualizada del proyecto.

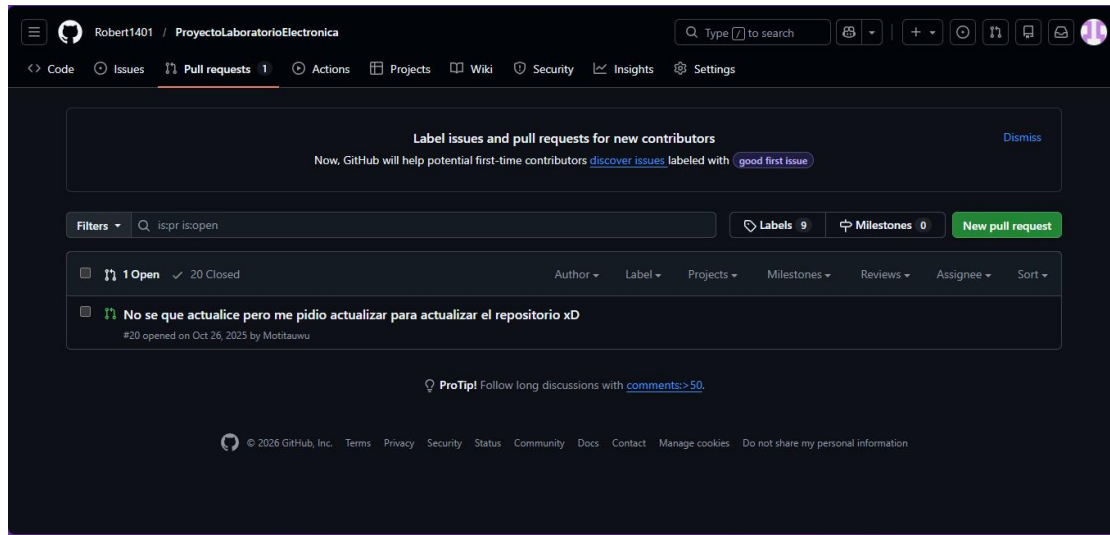
5. Funcionamiento del Pull Request

La opción de Pull Request funciona como un mecanismo de revisión antes de integrar cambios a la rama principal del proyecto. Primero se crea una rama independiente, se desarrollan mejoras o correcciones y posteriormente se envía una solicitud para fusionar esos cambios.

El Pull Request permite que otros miembros revisen el código, realicen comentarios y validen su calidad antes de aceptarlo. Este proceso reduce

errores, mejora la calidad del software y fomenta buenas prácticas de revisión colaborativa.

Es especialmente útil en proyectos donde participan varios desarrolladores simultáneamente.



6. Uso de Issues para organización del proyecto

Las Issues funcionan como un sistema interno de seguimiento de tareas, errores o propuestas de mejora.

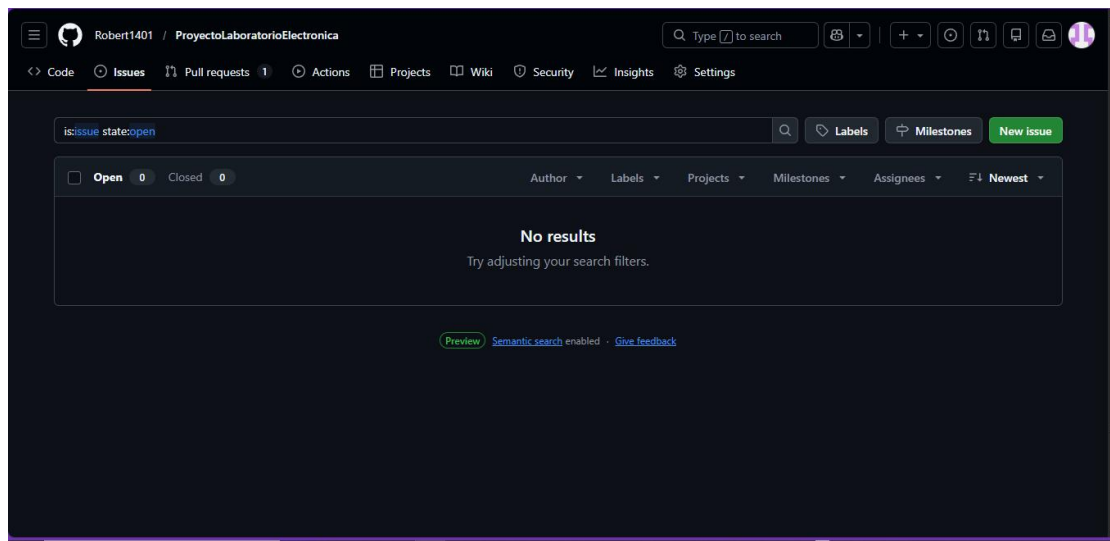
Durante la exploración noté que permiten asignar responsables, establecer etiquetas y llevar un historial de seguimiento. Esto convierte a GitHub en una herramienta no solo de control de versiones, sino también de gestión básica de proyectos.

En equipos académicos, las Issues pueden servir para dividir responsabilidades y dar seguimiento al cumplimiento de actividades.

7. Configuración de GitHub Pages

Para publicar un sitio web sencillo utilicé la función de GitHub Pages desde la sección de configuración del repositorio. Seleccioné la rama principal como

fuelle del sitio y el sistema generó automáticamente una dirección web pública.



Esta funcionalidad demuestra cómo GitHub puede integrarse en flujos de desarrollo web, permitiendo desplegar proyectos estáticos de manera rápida y gratuita.

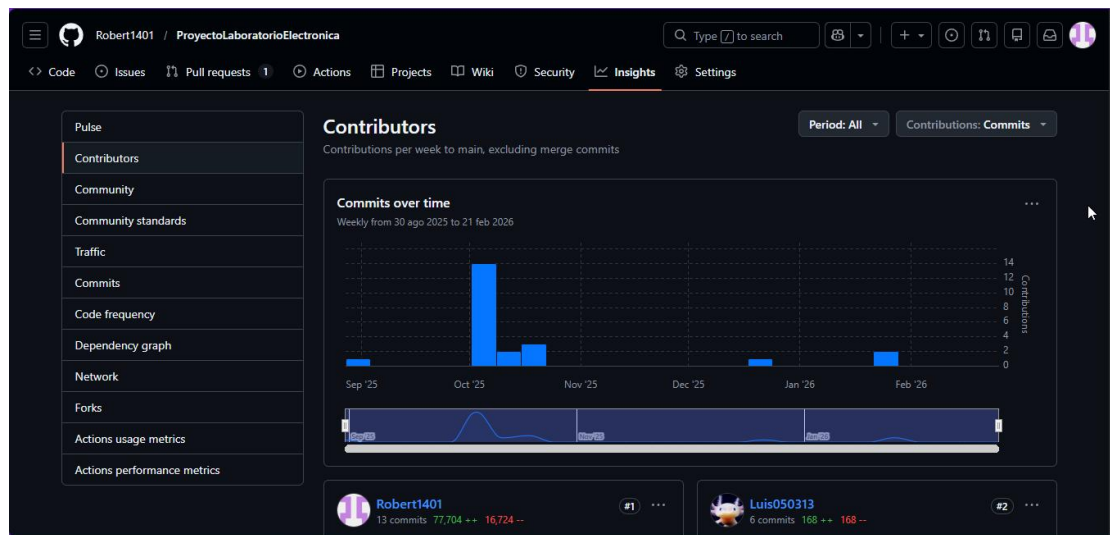
Resulta útil para portafolios, documentación técnica o demostraciones académicas.

8. Exploración de la sección Insights

En la sección Insights observé métricas relacionadas con la actividad del repositorio, como frecuencia de commits, contribuciones por usuario y evolución del proyecto en el tiempo.

Esta información puede servir a un profesor para evaluar participación individual en trabajos colaborativos, o a un equipo para medir productividad y detectar periodos de baja actividad.

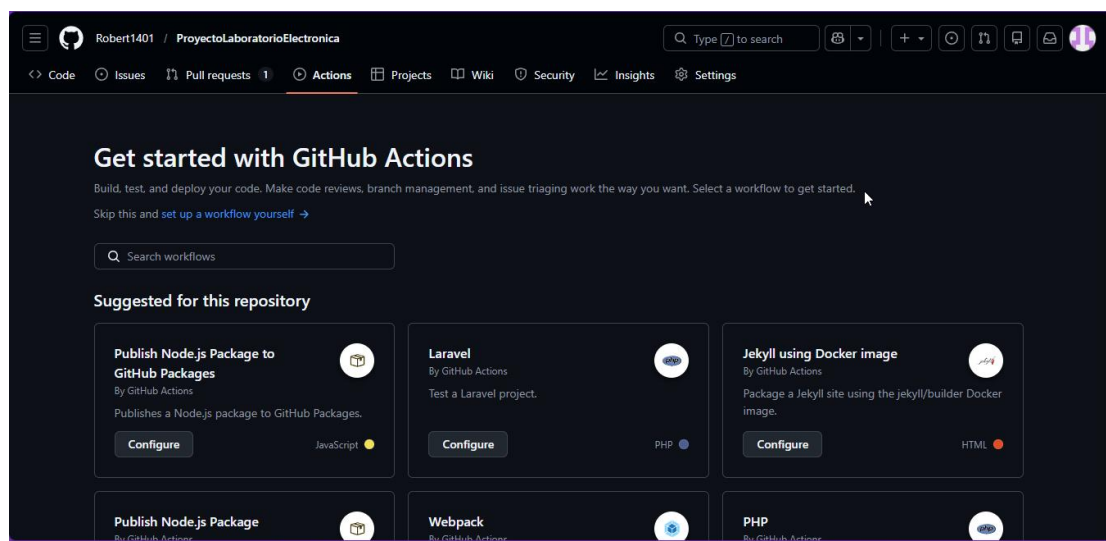
Desde un enfoque profesional, permite analizar patrones de desarrollo y mejorar la planificación.



9. Integración de GitHub Actions

GitHub Actions permite automatizar procesos mediante flujos de trabajo configurables. Durante la exploración entendí que pueden programarse tareas como pruebas automáticas, compilaciones o despliegues.

En proyectos en la nube, esta herramienta es fundamental para implementar prácticas de Integración Continua y Despliegue Continuo (CI/CD), reduciendo intervención manual y mejorando la confiabilidad del sistema.



10. Proyecto interesante encontrado

Al explorar la comunidad encontré proyectos colaborativos de gran escala que involucran múltiples desarrolladores de distintas partes del mundo. Lo que más llamó mi atención fue la estructura organizada de ramas, revisión de código y automatización de pruebas.

Este tipo de proyectos me inspira a aplicar buenas prácticas de documentación, control de versiones y automatización en mis propios desarrollos académicos, fortaleciendo mi preparación para entornos profesionales.

Conclusión

La exploración de GitHub permitió comprender que esta plataforma no solo funciona como almacenamiento de código, sino como un entorno completo de colaboración y control de versiones. Herramientas como los commits, las ramas, los pull requests y las issues facilitan la organización del trabajo en equipo, la revisión de cambios y el seguimiento del avance de un proyecto.

Asimismo, se observó que GitHub puede integrarse con procesos de automatización mediante GitHub Actions, lo que lo convierte en una herramienta clave dentro de metodologías modernas como DevOps e integración continua. En un contexto académico, el uso de GitHub ayuda a desarrollar habilidades que son ampliamente utilizadas en el ámbito profesional, como el trabajo colaborativo, la documentación del código y la gestión estructurada de proyectos.

Conocer y practicar con GitHub fortalece la preparación para entornos reales de desarrollo de software, permitiendo aplicar buenas prácticas que mejoran la organización, la calidad del código y la comunicación dentro de un equipo.

Referencias:

GitHub. (2024). GitHub Docs. <https://docs.github.com>